

PROYECTO 1

Jugador Autónomo para el Juego HEX

Estrategia basada en MC-RAVE con Union-Find Incremental

Documento de Estrategia Técnica

AUTORA

Alina María de la Noval Armenteros

Grupo C-311

Algoritmo : MC-RAVE (Gelly & Silver, 2011)

Framework : Monte Carlo Tree Search

Tablero : $N \times N$ hexagonal (even-r offset)

Límite de tiempo : 4,5 s por jugada

Índice

1	Introducción	2
2	Monte Carlo Tree Search	2
2.1	Descripción general	2
2.2	Selección — UCT	3
2.3	Expansión	3
2.4	Simulación (<code>_fast_rollout</code>)	4
2.5	Retropropagación	4
3	RAVE y MC-RAVE	4
3.1	Motivación: escasez de datos en las fases iniciales	4
3.2	Estadísticas AMAF	5
3.3	MC-RAVE: combinación de UCT y AMAF	5
4	Estrategia del Jugador Implementado	6
4.1	Arquitectura general	6
4.2	Union-Find incremental con nodos virtuales	7
4.3	Implementación MC-RAVE en <code>MCTSNode</code>	7
4.4	Restricción step-2: coherencia de las estadísticas AMAF	8
5	Estrategia Final del Jugador	9
5.1	Diagrama de flujo de <code>SmartPlayer.play()</code>	10
6	Conclusión	10

1 Introducción

El juego **HEX** es un juego de estrategia de dos jugadores en un tablero hexagonal de tamaño $N \times N$. El Jugador 1 (**rojo**) debe conectar el borde izquierdo con el derecho, mientras que el Jugador 2 (**azul**) debe conectar el borde superior con el inferior. Los jugadores alternan turnos colocando una ficha en cualquier celda vacía, y el primero en establecer una ruta continua entre sus bordes gana. El juego no puede terminar en empate [2].

Desde el punto de vista computacional, HEX presenta varias dificultades:

- **Espacio de estados exponencial.** Un tablero de 11×11 tiene $121!$ órdenes posibles de juego.
- **Sin función heurística fiable.** A diferencia del ajedrez, no existen evaluadores de posición bien establecidos.
- **Ramificación alta.** El factor de ramificación disminuye con el avance de la partida, pero es máximo al inicio, lo que hace inviable la búsqueda exhaustiva.

Por estas razones, los métodos de **Monte Carlo Tree Search** (MCTS) son especialmente adecuados para HEX. No requieren una función heurística elaborada, son *anytime* (producen una respuesta en cualquier momento), y su precisión mejora con el tiempo disponible [2]. De hecho, el programa MOHEX, basado en MCTS con RAVE, se convirtió en campeón mundial de HEX computacional en 2009 [2].

La estrategia implementada en este proyecto combina MC-RAVE con un Union-Find incremental, logrando reducir la complejidad de los rollouts de $O(N^4)$ a $O(N^2)$.

2 Monte Carlo Tree Search

2.1 Descripción general

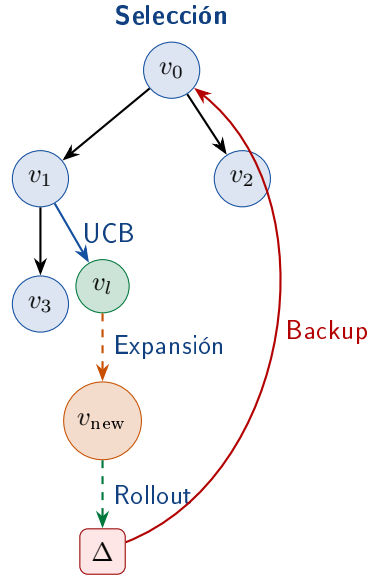
Monte Carlo Tree Search es un método de búsqueda que combina la estructura de un árbol de decisión con muestreo aleatorio de resultados [2]. El algoritmo itera sobre cuatro fases hasta agotar el presupuesto computacional, construyendo progresivamente un árbol sesgado hacia las regiones del espacio de búsqueda más prometedoras.

Algoritmo 1: MCTS — Iteración general (Browne et al. [1])

```

Input: Estado raíz  $s_0$ 
Output: Acción óptima  $a^*$ 
1 Crear nodo raíz  $v_0$  con estado  $s_0$ 
2 while dentro del presupuesto computacional do
3    $v_l \leftarrow \text{TreePolicy}(v_0)$                                 // Selección + Expansión
4    $\Delta \leftarrow \text{DefaultPolicy}(s(v_l))$                       // Simulación (rollout)
5    $\text{Backup}(v_l, \Delta)$                                        // Retropropagación
6 end
7 return  $a(\text{BestChild}(v_0, \theta))$ 
```

La figura siguiente ilustra las cuatro fases en una sola iteración:



2.2 Selección — UCT

Durante la fase de selección, el árbol se recorre desde la raíz hasta alcanzar un nodo con movimientos no explorados, eligiendo en cada nivel el hijo con mayor valor de acuerdo a la política UCB1 (*Upper Confidence Bound*) [2].

Definición: UCT — selección en árbol (Gelly & Silver [1]; Browne et al. [2])

Para un nodo v con padre $\text{padre}(v)$, el valor UCT se define como:

$$\text{UCT}(v) = \underbrace{\frac{Q(v)}{N(v)}}_{\text{explotación}} + C_p \underbrace{\sqrt{\frac{\ln N(\text{padre}(v))}{N(v)}}}_{\text{exploración}} \quad (1)$$

donde:

- $Q(v)$ — suma acumulada de victorias en el nodo v ; $Q(v)/N(v)$ es la tasa de victorias.
- $N(v)$ — número de visitas al nodo v .
- $N(\text{padre}(v))$ — número de visitas al padre de v .
- $C_p \geq 0$ — constante de exploración (`EXPLORATION_C` en el código).

El primer término favorece la **explotación**: los movimientos con mayor tasa de victorias. El segundo término favorece la **exploración**: su valor decrece a medida que $N(v)$ crece, incentivando la visita a nodos poco explorados. Cuando $N(v) = 0$ el valor es $+\infty$, garantizando que todo hijo sea visitado al menos una vez [2].

En la implementación, `EXPLORATION_C = 0.0`, es decir, el término de exploración UCT está desactivado. Browne et al. señalan explícitamente que, para Go y HEX, cuando se emplean heurísticas de historia como RAVE y AMAF, se ha observado que este constante debe ser cero [2], pues las estadísticas AMAF ya proveen exploración implícita. Este punto se retoma en la Sección 3.3.

2.3 Expansión

Cuando el algoritmo de selección alcanza un nodo v que no está *completamente expandido* (es decir, tiene al menos un movimiento no explorado), se selecciona aleatoriamente uno de esos movimientos sin probar, $a \in A(s(v))$, y se añade un nuevo hijo v' al árbol:

$$s(v') = f(s(v), a), \quad a(v') = a.$$

En la implementación, la lista de movimientos legales se hereda incrementalmente del padre (filtro $O(N)$), evitando la reconstrucción costosa $O(N^2)$ del tablero completo en cada expansión.

2.4 Simulación (`_fast_rollout`)

A partir del nodo recién expandido, se ejecuta una simulación hasta alcanzar un estado terminal. La implementación concreta es la función `_fast_rollout`, cuya lógica es la siguiente:

1. Se copia el tablero actual (`sim_grid`).
2. Se inicializan **dos estructuras Union-Find** —una por jugador— a partir del estado copiado, con coste $O(N^2)$ una sola vez por simulación (`_build_uf_from_grid`).
3. La lista de celdas vacías `node.empty` se mezcla aleatoriamente (`random.shuffle`).
4. Las celdas se asignan en orden, alternando entre el jugador actual y el oponente. Tras cada asignación se llama a `uf.place()` —coste $O(\alpha)$ amortizado— y se comprueba inmediatamente si ese jugador ha ganado (`uf1.p1_wins()` / `uf2.p2_wins()`). En cuanto se detecta victoria se interrumpe el bucle.
5. Se devuelve el ganador $w \in \{1, 2\}$ y el diccionario `moves_by_player` con los movimientos realizados por cada jugador durante el rollout (necesario para actualizar las estadísticas AMAF).

El resultado $w \in \{1, 2\}$ indica qué jugador ganó la simulación. Junto con `moves_by_player` —diccionario que registra qué movimientos realizó cada jugador durante el rollout— se pasa a la fase de retropropagación para actualizar tanto las estadísticas UCT como las estadísticas AMAF de los nodos del árbol.

2.5 Retropropagación

Una vez obtenido el ganador w del rollout, se actualizan las estadísticas de todos los nodos en el camino desde la hoja hasta la raíz:

$$N(v) \leftarrow N(v) + 1, \tag{2}$$

$$Q(v) \leftarrow Q(v) + \mathbf{1}[\text{player}(v) = w], \tag{3}$$

donde $\mathbf{1}[\cdot]$ es la función indicadora: se suma 1 a $Q(v)$ únicamente si el jugador que realizó el movimiento en el nodo v es el ganador w . La tasa de victorias estimada es entonces $Q(v)/N(v)$.

Perspectiva *player-centric*. Cada nodo almacena el identificador del jugador que realizó el movimiento (`node.player`). Esto simplifica la retropropagación: no es necesario invertir el signo de la recompensa al subir por el árbol, ya que cada nodo acumula victorias solo desde su propia perspectiva. El nodo raíz tiene `player=None` y por tanto nunca acumula victorias —solo visitas— lo cual es correcto, pues únicamente se usa $N(\text{raíz})$ como denominador del término de exploración en los hijos.

3 RAVE y MC-RAVE

3.1 Motivación: escasez de datos en las fases iniciales

Un problema central de MCTS puro es que las estadísticas de los nodos cercanos a la raíz son escasas durante las primeras iteraciones. Un movimiento puede no haberse visitado lo suficiente

para que su estimación $Q(v)/N(v)$ sea fiable, lo que conduce a decisiones subóptimas al inicio de la búsqueda.

El algoritmo **RAVE** (*Rapid Action Value Estimation*) [1] propone aprovechar la heurística AMAF (*All Moves As First*): si una acción a apareció en alguna simulación posterior al estado s , esa información también es relevante para estimar el valor de a en s , independientemente del turno exacto en que se jugó.

3.2 Estadísticas AMAF

Definición: valor AMAF

El valor AMAF de la acción a en el estado s se define como:

$$\tilde{Q}(s, a) = \frac{\tilde{W}(s, a)}{\tilde{N}(s, a)} \quad (4)$$

donde:

- $\tilde{W}(s, a)$ — número de victorias en simulaciones donde a fue jugado en cualquier turno posterior a s .
- $\tilde{N}(s, a)$ — número de simulaciones donde a fue jugado en cualquier turno posterior a s .

La hipótesis subyacente es que, en juegos de posición como HEX, el valor de una jugada no depende fuertemente del turno exacto en que se realiza [1]. Las estadísticas AMAF acumulan evidencia mucho más rápido que las estadísticas UCT clásicas, aunque introducen un sesgo sistemático.

3.3 MC-RAVE: combinación de UCT y AMAF

El algoritmo **MC-RAVE** [1] combina el valor UCT $Q(s, a)$ con el valor AMAF $\tilde{Q}(s, a)$ mediante una ponderación que varía dinámicamente con el número de visitas:

Definición: valor combinado MC-RAVE (Gelly & Silver [1])

$$Q^*(s, a) = (1 - \beta) Q(s, a) + \beta \tilde{Q}(s, a) \quad (5)$$

con la ponderación mínima-MSE:

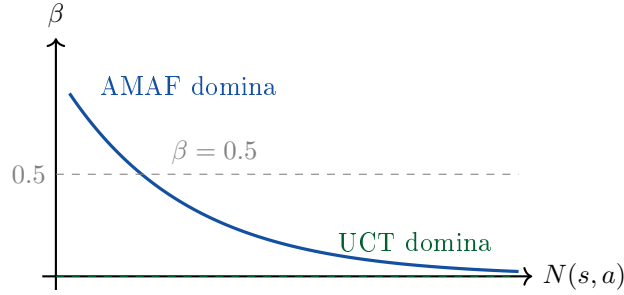
$$\beta(s, a) = \frac{\tilde{N}(s, a)}{N(s, a) + \tilde{N}(s, a) + 4 N(s, a) \tilde{N}(s, a) b^2} \quad (6)$$

donde b es el parámetro de sesgo RAVE (RAVE_BIAS en el código).

Comportamiento dinámico de β . Cuando $N(s, a)$ y $\tilde{N}(s, a)$ son pequeños (inicio de la búsqueda), $\beta \approx 1$ y el valor combinado está dominado por la estimación AMAF, que es menos precisa pero tiene mucha más evidencia. A medida que el árbol crece y $N(s, a) \rightarrow \infty$, el denominador de (6) crece cuadráticamente y $\beta \rightarrow 0$, de modo que el valor UCT — más preciso pero más lento — termina dominando. Este mecanismo garantiza que la convergencia asintótica de MC-RAVE sea equivalente a la de UCT [1].

Parámetro k y C_p . El parámetro b en (6) se relaciona con el parámetro de equivalencia k de Gelly & Silver mediante $k = 1/(4b^2)$ [1]. En la implementación se usa $b = 0,00913$, equivalente a $k \approx 3000$, determinado empíricamente para tableros de 11×11 [1]. Browne et al. señalan que

para Go y HEX, cuando se emplean heurísticas como RAVE y AMAF, se ha observado que C_p debe ser cero [2], pues estas heurísticas ya proveen una forma de exploración implícita que hace innecesario el término UCT. La implementación establece `EXPLORATION_C = 0.0` en consonancia con esto.



La figura ilustra cómo β decrece monótonicamente con $N(s, a)$: al inicio de la búsqueda las estadísticas AMAF guían la selección, mientras que con suficientes visitas el estimador UCT toma el control.

4 Estrategia del Jugador Implementado

4.1 Arquitectura general

La implementación se organiza en tres componentes principales, cuya relación se muestra en la Figura 1.

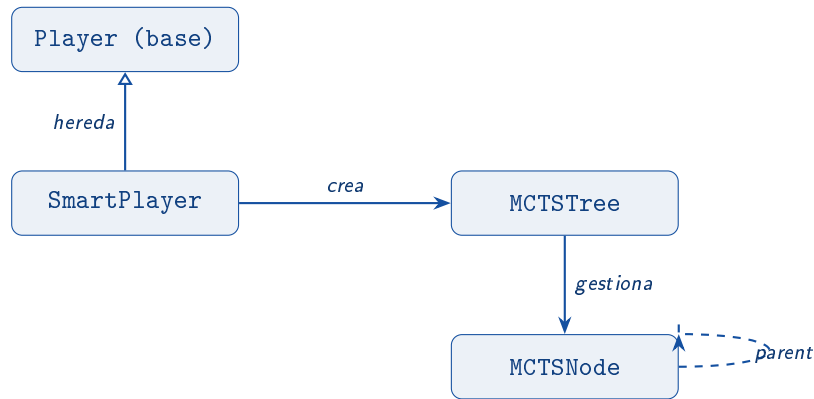


Figura 1: Relación entre las clases de la implementación.

SmartPlayer. Clase principal que implementa la interfaz **Player**. Encapsula la lógica de decisión en cascada: evalúa movimientos decisivos/anti-decisivos mediante BFS (`check_win`) y, si no aplica ninguna regla directa, delega toda la búsqueda al motor **MCTSTree**.

MCTSTree. Motor de búsqueda MC-RAVE. Mantiene la raíz del árbol, gestiona el presupuesto de tiempo y ejecuta el bucle de iteraciones. Internamente, cada iteración invoca `_fast_rollout`, que crea y utiliza instancias de **HexUnionFind** para detectar victorias durante la simulación en $O(\alpha)$.

MCTSNode. Nodo individual del árbol. Almacena las estadísticas UCT (`visits`, `wins`) y AMAF (`visits_amaf`, `wins_amaf`), la lista incremental de celdas vacías (`empty`), los movimientos sin explorar (`untried_moves`), y el puntero al nodo padre (`parent`).

4.2 Union-Find incremental con nodos virtuales

Una optimización crítica de la implementación es el uso de Union-Find (DSU) incremental basado en las técnicas de Tarjan [4] (path halving + union by rank) y la idea de nodos virtuales de borde de Anshelevich [3].

La clase `HexUnionFind` trabaja con índices lineales. Dado un tablero de tamaño `size`, se definen $n = \text{size} * \text{size}$ nodos reales (uno por celda, con índice $r * \text{size} + c$) y cuatro **nodos virtuales** adicionales que representan los bordes del tablero:

Variable	Índice	Representa	Jugador
VL	n	Borde izquierdo (columna 0)	1
VR	$n+1$	Borde derecho (columna $\text{size}-1$)	1
VT	$n+2$	Borde superior (fila 0)	2
VB	$n+3$	Borde inferior (fila $\text{size}-1$)	2

Cuando se coloca una ficha del Jugador 1 en la columna 0, se ejecuta `union(i, VL)`; en la columna $\text{size}-1$, `union(i, VR)`. La condición de victoria del Jugador 1 se reduce entonces a comprobar si `find(VL) == find(VR)`, operación de coste $O(\alpha)$ amortizado [4], donde α es la función inversa de Ackermann, prácticamente constante. Análogamente para el Jugador 2 con VT y VB.

El método `place(r, c, player, grid)` une la celda recién colocada con sus vecinos del mismo color ya presentes en el tablero, además de con el nodo virtual de borde si corresponde. Toda la detección de victoria queda así en tiempo $O(\alpha)$ por movimiento, sin necesidad de recorrer el tablero con BFS.

4.3 Implementación MC-RAVE en MCTSNode

Cada nodo almacena cuatro contadores:

Atributo	Tipo	Descripción
<code>visits</code>	<code>int</code>	$N(v)$ — visitas UCT
<code>wins</code>	<code>float</code>	$W(v)$ — victorias UCT
<code>visits_amaf</code>	<code>int</code>	$\tilde{N}(v)$ — visitas AMAF
<code>wins_amaf</code>	<code>float</code>	$\tilde{W}(v)$ — victorias AMAF

El valor de selección se calcula en el método `rave_value()`, que implementa la ecuación (5):

Cálculo de `rave_value()` en `MCTSNode`

```

1 if visits = 0 then
2   if visits_amaf > 0 then
3     return  $\tilde{W}/\tilde{N}$  + FPU           // FPU = 0.35, First Play Urgency
4   return  $+\infty$                  // Nodo no visitado: prioridad máxima
5  $q_{\text{uct}} \leftarrow W/N$ 
6 if visits_amaf > 0 then
7    $q_{\text{amaf}} \leftarrow \tilde{W}/\tilde{N}$ 
8    $\beta \leftarrow \tilde{N}/(N + \tilde{N} + 4 N \tilde{N} b^2)$ 
9    $q \leftarrow (1 - \beta) q_{\text{uct}} + \beta q_{\text{amaf}}$ 
10 else
11    $q \leftarrow q_{\text{uct}}$ 
12 return  $q + C_p \sqrt{\ln N_{\text{padre}}/N}$ 

```

El parámetro **FPU** (*First Play Urgency*) = 0,35, propuesto originalmente por Gelly et al. y descrito en Browne et al. [2], asigna un valor provisional a nodos que tienen estadísticas AMAF disponibles pero todavía no han sido visitados directamente por la política de árbol (**visits**=0). En ese caso, la función **rave_value()** devuelve $\tilde{W}/\tilde{N} + \text{FPU}$, elevando artificialmente la urgencia de visitar ese nodo. Si un nodo no tiene ningún tipo de estadísticas (ni UCT ni AMAF), se le asigna $+\infty$ para garantizar que todo movimiento sea explorado al menos una vez antes de que el UCB1 entre en acción.

4.4 Restricción step-2: coherencia de las estadísticas AMAF

Una decisión de diseño fundamental en MC-RAVE es que las estadísticas AMAF *solo se actualizan para movimientos del mismo jugador*. Esto refleja el requisito de Gelly & Silver [1]: la actualización AMAF de (s_t, a_u) se realiza únicamente cuando $u \geq t$ y *el turno en u corresponde al mismo jugador que en t* (condición $u \equiv t \pmod{2}$).

En la implementación, la retropropagación itera desde la hoja hasta la raíz; en cada nodo con hijos, determina el jugador que jugaría a continuación (**next_player**) y actualiza **visits_amaf** / **wins_amaf** solo en los hijos que corresponden a movimientos realizados por ese jugador durante el rollout:

Retropropagación con actualización AMAF (step-2 constraint)

Input: Nodo hoja v_ℓ , ganador w , movimientos del rollout $M[1], M[2]$

```

1  $v \leftarrow v_\ell$ 
2 while  $v \neq \text{None}$  do
3    $v.\text{visits} += 1$ 
4   if  $v.\text{player} = w$  then
5      $v.\text{wins} += 1$ 
6   if  $v$  tiene hijos then
7      $p_{\text{next}} \leftarrow$  siguiente jugador tras  $v$ 
8     for cada movimiento  $m \in M[p_{\text{next}}]$  do
9       if  $m \in v.\text{children}$  then
10         $v.\text{children}[m].\text{visits\_amaf} += 1$ 
11        if  $v.\text{children}[m].\text{player} = w$  then
12           $v.\text{children}[m].\text{wins\_amaf} += 1$ 
13    $v \leftarrow v.\text{parent}$ 

```

5 Estrategia Final del Jugador

La lógica del método `SmartPlayer.play()` implementa una **estrategia en cascada** que combina reglas deterministas simples con la búsqueda MC-RAVE:

Estrategia en cascada de SmartPlayer

1. **Un solo movimiento legal.** Si solo hay una celda vacía, se juega ahí directamente.
2. **Primer movimiento (tablero vacío).** Se juega en el centro ($\lfloor N/2 \rfloor, \lfloor N/2 \rfloor$), posición estratégicamente ventajosa en HEX.
3. **Movimiento decisivo.** Se comprueba si existe un movimiento que gane la partida inmediatamente. Si existe, se juega ahí.
4. **Movimiento anti-decisivo.** Se comprueba si el oponente puede ganar en su siguiente turno y, en ese caso, se bloquea esa jugada.
5. **MC-RAVE durante 4,5 segundos.** Se ejecuta el motor MC-RAVE con el presupuesto de tiempo. Al finalizar, se selecciona el movimiento más robusto.

Selección del hijo más visitado. Tras el presupuesto de búsqueda, la acción seleccionada es la del **hijo más visitado** de la raíz (*robust child* según la terminología de Browne et al. [2]), no necesariamente el de mayor valor esperado. Esta elección es más *robusta* estadísticamente: el hijo más visitado tiene las estimaciones más fiables y concentra la mayor parte de la evidencia de simulación. En cambio, el hijo de mayor Q/N puede corresponder a un nodo con pocas visitas y alta varianza [2].

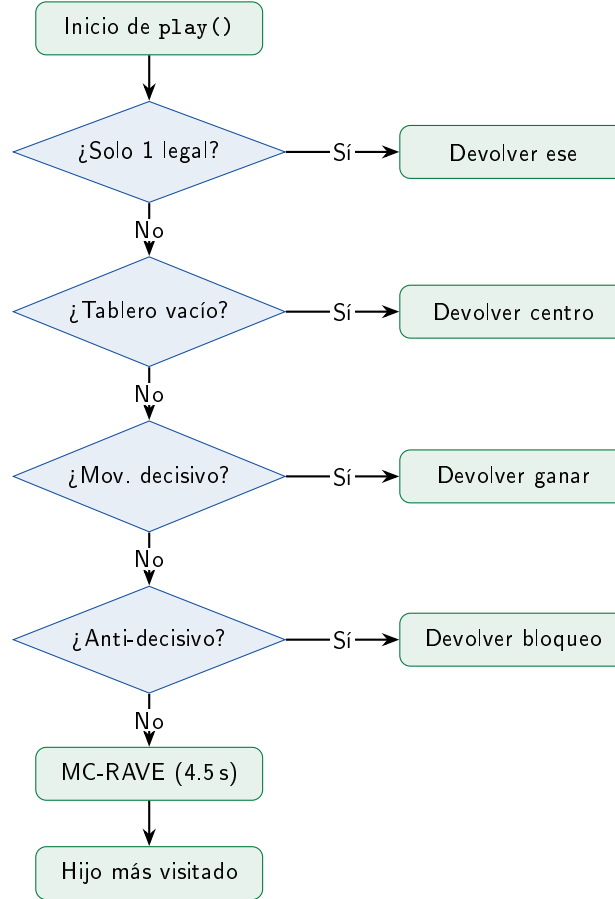
Formalmente, la acción final es:

$$a^* = \arg \max_{a \in A(s_0)} N(\text{hijo}(v_0, a)).$$

Movimientos decisivos y anti-decisivos. Las comprobaciones de los pasos 3 y 4 usan BFS ($O(N^2)$), lo cual es aceptable pues solo se ejecutan una vez por turno antes de iniciar MCTS, y su detección temprana puede ahorrar el tiempo completo de búsqueda. Browne et al. [2] clasifican esta técnica bajo el nombre de *decisive moves* dentro de las mejoras de la política de árbol.

Gestión del tiempo. El límite de tiempo de 4,5s es inferior al límite de descalificación de 5 s establecido por las reglas del torneo, proporcionando un margen de seguridad para la inicialización del árbol y las comprobaciones de movimientos decisivos/anti-decisivos.

5.1 Diagrama de flujo de SmartPlayer.play()



6 Conclusión

Se ha presentado el diseño e implementación de un jugador autónomo para el juego HEX basado en el algoritmo MC-RAVE [1], integrado con un Union-Find incremental que reduce la complejidad computacional de los rollouts de $O(N^4)$ a $O(N^2)$.

Los elementos principales de la estrategia son:

- **MC-RAVE con $C_p = 0$ y $k \approx 3000$.** La combinación dinámica de estadísticas UCT y AMAF permite aprender más rápido en las fases tempranas sin sacrificar la convergencia asintótica a la política óptima. El parámetro $C_p = 0$ suprime el término de exploración UCT, que resulta redundante cuando RAVE ya provee exploración implícita [2].
- **Union-Find con nodos virtuales.** Detección de victoria en $O(\alpha)$ por movimiento, maximizando el número de iteraciones MCTS dentro del límite de tiempo.
- **Lista de vacíos incremental.** Herencia $O(N)$ en la expansión de nodos, evitando la reconstrucción $O(N^2)$ del tablero en cada iteración.
- **Cascada de reglas deterministas.** Las comprobaciones de movimientos decisivos y anti-decisivos garantizan el comportamiento correcto en situaciones críticas antes de delegar en MCTS.
- **Selección del hijo más visitado.** Elección estadísticamente robusta que prefiere la fiabilidad de las estimaciones sobre el valor esperado puntual [2].

La integración de estas técnicas produce un jugador competitivo dentro del límite de 5s por jugada establecido en las reglas del torneo.

Referencias

- [1] S. Gelly and D. Silver, Monte-Carlo Tree Search and Rapid Action Value Estimation in Computer Go, *Artificial Intelligence*, 175(11):1856–1875, 2011.
- [2] C. Browne, E. Powley, D. Whitehouse, S. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, A Survey of Monte Carlo Tree Search Methods, *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [3] V. Anshelevich, A hierarchical approach to computer Hex, *Artificial Intelligence*, 134(1-2):101–120, 2002.
- [4] R. E. Tarjan, Efficiency of a good but not linear set union algorithm, *Journal of the ACM*, 22(2):215–225, 1975.